CS-781 Capstone, Phase 3: Rhetorical Role Prediction

A hierarchical sentence tagger on the four Phase-1 language models

Manish Kumar (251330604)
CS-781, IIT Kanpur

October 2026

Abstract

Every sentence of a Corporate Law (CL) or Information Technology (IT) judgment is assigned one of thirteen rhetorical roles. Labels come in long runs (the next sentence has the same label with probability 0.94 in CL and 0.75 in IT), so we build a hierarchical tagger: a Phase-1 language model encodes windows of neighbouring sentences, each sentence is mean-pooled, a BiLSTM with position and domain embeddings reads the document, and a sentence-level CRF decodes the label sequence. All four Phase-1 architectures are adapted with nine techniques (LoRMA+, LoRA, rsLoRA, QLoRA, AdaLoRA, IA^3, prefix tuning, prompt tuning and knowledge distillation), evaluated by five-fold document-level cross-validation over the 90 labelled documents. The context-free ablation scores 0.361 out-of-fold macro-F1 against 0.455 for the same encoder with context. A tuning round (causal attention for the decoder backbones, longer training, weaker logit adjustment) lifts every technique it was applied to, the best single models reaching 0.501 (A1 LoRA) and 0.500 (A4 LoRMA+); a four-model ensemble reaches 0.511. All numbers are generated from the run outputs by `p3/report/make_figures.py`.

Contents

1 Introduction **2**

2 Data **2**

3 Methods **3**
3.1 Hierarchical model . . . 3
3.2 Backbones . . . 3
3.3 Training . . . 4
3.4 Adaptation methods . . . 4

4 Evaluation and submission protocol **5**

5 Results **5**
5.1 First round . . . 5
5.2 Tuning rounds . . . 6
5.3 Per-label results and confusions . . . 8
5.4 Ensembles, calibration and post-processing . . . 9
5.5 Submissions . . . 10

6 Analysis **11**

7 Limitations and next steps **11**

8 Conclusions **11**

9 Reproducibility **12**

1 Introduction

Rhetorical role (RR) prediction labels each sentence of a judgment with its function: facts, issues, the arguments of each side, statutes, precedents relied on or not, the lower court's ruling, the ratio of the decision, the present court's ruling, dissent, or none. Phase 3 provides judgments from two domains, Corporate Law (CL) and Information Technology (IT); the prediction must contain exactly one official label per sentence, in order, and the primary metric is macro-F1 (macro precision and recall are secondary). The rules require the Phase-1 models as the only pretrained weights, all four architectures across the submissions, at least six substantially different experiments of which at least five are different adaptation techniques, LoRMA and knowledge distillation.

The report covers the data (Section 2), the model and every adaptation method (Section 3), the evaluation and submission protocol (Section 4), the first round of experiments (Section 5.1), the tuning rounds (Section 5.2), per-label and confusion results (Section 5.3), and ensembles, calibration and decoding-time post-processing (Section 5.4).

2 Data

Table 1: Documents and sentences (train + official dev, which we pool for cross-validation). Persistence = probability that a sentence has the same label as the previous one.

Domain	Docs	Sentences	Median sent./doc	Max	Test docs	Test sent.	Persistence
CL	45	11,989	123	1,816	5	1,339	0.938
IT	45	6,998	123	442	5	858	0.754

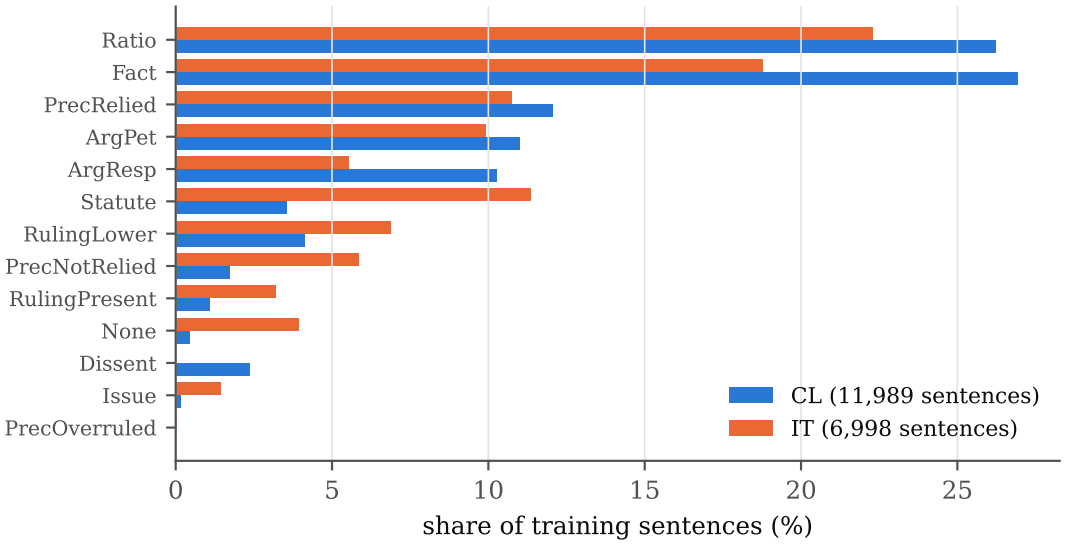


Figure 1: Label distribution per domain (train + dev).

Table 2: Sentences per label and domain.

Label	CL		IT	
Fact	3,228	26.9%	1,314	18.8%
Issue	20	0.2%	102	1.5%
ArgumentPetitioner	1,320	11.0%	693	9.9%
ArgumentRespondent	1,232	10.3%	388	5.5%
Statute	425	3.5%	794	11.3%
Dissent	284	2.4%	0	0.0%
PrecedentReliedUpon	1,447	12.1%	752	10.7%
PrecedentNotReliedUpon	209	1.7%	410	5.9%
PrecedentOverruled	0	0.0%	3	0.0%
RulingByLowerCourt	494	4.1%	482	6.9%
RatioOfTheDecision	3,144	26.2%	1,560	22.3%
RulingByPresentCourt	131	1.1%	225	3.2%
None	55	0.5%	275	3.9%

Dissent occurs in 2 of the 90 documents and PrecedentOverruled in 1 (3 sentences).

3 Methods

3.1 Hierarchical model

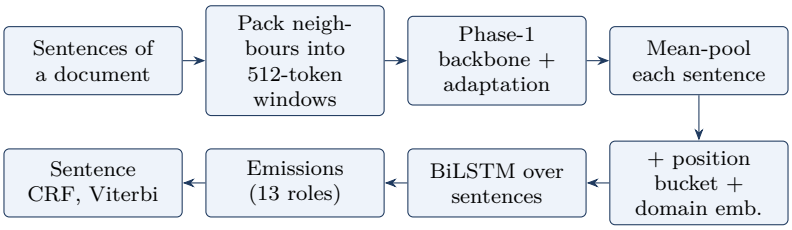


Figure 2: The rhetorical-role model (`p3/rr/model.py`).

- **Sentence encoding with context.** Each sentence (at most 128 tokens; the 95th percentile is 72–84 words) is tokenized with the Phase-1 BPE as `" "` + text, and consecutive sentences are packed into windows of at most 512 tokens, so every sentence is encoded together with its neighbours. The ablation E0 encodes every sentence alone (`data.context: none`).
- **Pooling.** A sentence vector is the mean of its tokens' final hidden states.
- **Document layer.** The vector is projected to 256 dimensions (dropout 0.1, GELU), added to a relative-position embedding (sentence index / document length, 20 buckets) and a domain embedding (CL or IT), and a one-layer BiLSTM [5] reads the sentences of the document.
- **Output.** A linear layer gives 13 emission scores per sentence; a linear-chain CRF [8] over the sentence sequence learns label transitions and is decoded with Viterbi. E0 uses independent softmax decisions instead.

The design follows hierarchical BiLSTM-CRF models for rhetorical roles [1, 12].

3.2 Backbones

The backbone loading and the A2 handling are shared with Phase 2 (`p2/lner/backbones.py`): A1 and A4 run with the Phase-1 causal mask (`attention: causal`) or without it (`bidirectional`); A3 reads the whole window as its bidirectional prefix; A2 uses its decoder states (its encoder output is nearly collapsed, Phase-2 report). The first-round experiments use bidirectional attention for A1

and A4, as in Phase 2; the tuning rounds switch them to causal (Section 5.2). All experiments start from the cleaned Phase-1 checkpoints (1B tokens); E1 is also run on the original A4 checkpoint.

3.3 Training

Each optimizer step accumulates 4 chunks of 128 consecutive sentences (random start per chunk; every document contributes $\lceil n/128 \rceil$ chunks per epoch). AdamW, learning rate 10^{-3} for the adapted backbone parameters (3×10^{-3} for IA^3 and prompt tuning, 10^{-4} for the fully fine-tuned KD student) and 10^{-3} for the document layer, weight decay 0.01, 10% linear warm-up then cosine decay, gradient clipping 1.0, bf16. First round: up to 12 epochs, early stopping after 4 epochs without improvement on the fold's held-out documents; tuned runs: 20 epochs, patience 6. The best epoch is kept.

Logit adjustment. The training loss uses scores $s_y + \tau \log \pi_y$, where π is the label prior of the training folds [13]; at inference the raw scores are used. This raises the relative score of rare roles, which macro-F1 weights equally. $\tau = 1$ in the first round, 0.5 after tuning.

3.4 Adaptation methods

LoRA, LoRMA+ and IA^3 come from Phase 2, in `p2/lner/peft.py`. The other methods are new, in `p3/rr/peft_extra.py`. The document layer and CRF are always trained.

- **LoRMA+** [2]: $W = (I + \frac{\alpha}{r} BA)\, W_0$, computed as $h_0 = W_0 x$, $h = h_0 + \frac{\alpha}{r} B(Ah_0)$; $r = 16$, $\alpha = 32$, on the attention and MLP projections.
- **LoRA** [6]: $h = W_0 x + \frac{\alpha}{r} BAx$; $r = 16$, $\alpha = 32$.
- **rsLoRA** [7]: LoRA with scale $\alpha/\sqrt{r}$ instead of α/r, which keeps the update's size stable at high rank; $r = 64$, $\alpha = 16$.
- **QLoRA** [3]: every linear layer of the frozen backbone is stored as 4-bit NormalFloat (the 16 quantiles of a standard normal rescaled to $[-1, 1]$), with one absmax scale per block of 64 weights and two 4-bit codes packed per byte; weights are de-quantized on the fly, and LoRA ($r = 16$) is trained in higher precision on top. The relative reconstruction error of a Gaussian weight matrix is 9%, and storage is 8× smaller than fp32.
- **AdaLoRA** [14]: the update is $P\,\mathrm{diag}(\lambda)\,Q$ with an orthogonality penalty $\|P^\top P - I\|^2 + \|QQ^\top - I\|^2$ (weight 0.1). The importance of each singular value is its smoothed sensitivity $|w \cdot \nabla w|$ times its uncertainty (EMA factors 0.85), summed over λ_i and the mean of its column of P and row of Q. The global budget falls cubically from rank 12 to an average of 8 per module between 10% and 70% of training; the lowest-importance singular values are masked every 10 steps.
- **IA^3** [11]: learned scaling vectors on keys, values and the MLP's inner activation, initialised to one.
- **Prefix tuning** [10]: 16 learned key/value vectors per self-attention layer, produced by a shared two-layer MLP from 16 prefix embeddings (hidden 512, last layer initialised to zero); prefix keys carry no rotary position, and every attention mask is extended to let all tokens attend to them.
- **Prompt tuning** [9]: 20 learned input embeddings, initialised from random vocabulary embeddings, replace the first 20 positions of every window.
- **Knowledge distillation** [4]: the student is A4 with 6 of its 10 blocks (blocks 0, 2, 4, 5, 6, 8, which keep all three global-attention layers), 62.9M parameters against 90.3M for the teacher, fully fine-tuned with $\alpha\,\mathcal{L}_{\text{task}} + (1-\alpha)\,T^2\,\mathrm{KL}(p^T_{\text{teacher}} \,\|\, p^T_{\text{student}})$ on the emission scores, $\alpha = 0.5$, $T = 2$. The fold-k student learns from the fold-k teacher, which was trained on the same documents.

4 Evaluation and submission protocol

- **Cross-validation.** The official dev set has only 5 documents per domain, with a different label mix from train (for example ArgumentPetitioner is 9% of CL train and 25% of CL dev). We pool train and dev (90 documents) and split them into 5 document-level folds, stratified by domain (`cv.seed`=0, the same folds for every run).
- **Out-of-fold (OOF) score.** Each document is predicted by the fold model that did not train on it; macro-F1 is computed over all 90 documents (labels present in gold or predictions, as in scikit-learn), and also per domain. The mean ± standard deviation of the five per-fold scores is reported alongside.
- **Submission.** The five fold models score the test documents; their log-softmax emissions and CRF parameters are averaged and decoded with Viterbi (`make_submission.py`). The writer checks that every test document is present, with one official label per sentence in order, and puts `prediction.json` at the root of the zip.
- **Cross-run ensembles** (`ensemble_cv.py`, `calibrate.py`): the same averaging over several experiments, scored out-of-fold with each fold's models.
- **Calibration** (`calibrate.py`): a per-label bias added to the emissions before Viterbi, fitted by coordinate ascent over $\{-1, -0.75, \ldots, 1\}$ to maximise macro-F1 (labels with fewer than 20 out-of-fold sentences keep zero). The reported score is nested: for each fold, the bias is fitted on the other folds' documents only.
- **Decoding-time post-processing** (`postproc.py`): never emitting given labels, and scaling the CRF transition scores. Because a test set has only 10 documents, it is also scored on 3,000 simulated test sets of 5 CL + 5 IT out-of-fold documents, each scored on its own.

5 Results

5.1 First round

Table 3: First-round experiments: out-of-fold (OOF) macro-F1 over 90 documents, per domain, fold mean ± std, macro precision / recall, minutes per fold, peak GPU memory (GB), checkpoint (MB).

Run	Arch	Method	Trainable	OOF F1	CL	IT	Fold mean	P	R	Min	GB	MB
E1_a4_lorma	A4	lorma	2.81M	0.455	0.462	0.468	0.478±0.028	0.438	0.509	9.3	4.6	11.3
E1_a4_lorma_origP1	A4	lorma	2.81M	0.427	0.434	0.403	0.437±0.021	0.412	0.495	9.5	4.6	11.3
E0_a4_lorma_sentonly	A4	lorma	2.42M	0.361	0.324	0.378	0.360±0.019	0.340	0.412	25.5	9.1	9.7
E2_a1_lora	A1	lora	2.37M	0.330	0.347	0.310	0.335±0.033	0.324	0.377	6.0	3.2	9.5
E3_a3_rslora	A3	rslora	7.68M	0.441	0.458	0.461	0.473±0.025	0.433	0.493	7.3	3.5	30.7
E4_a2_ia3	A2	ia3	0.62M	0.406	0.417	0.417	0.415±0.029	0.397	0.482	7.1	3.2	2.5
E5_a4_qlora	A4	qlora	2.77M	0.423	0.439	0.424	0.458±0.018	0.405	0.485	9.8	4.3	11.1
E6_kd_a4_small	A4	full (KD)	62.94M	0.403	0.412	0.385	0.434±0.026	0.392	0.449	14.4	4.1	251.8
E7_a3_adalora	A3	adalora	1.93M	0.396	0.414	0.393	0.416±0.030	0.379	0.476	8.5	3.4	7.8
E8_a4_prefix	A4	prefix	3.63M	0.326	0.354	0.285	0.331±0.028	0.322	0.409	7.5	2.9	14.5
E9_a1_prompt	A1	prompt	0.62M	0.272	0.303	0.236	0.287±0.035	0.277	0.353	4.9	2.0	2.5

- **Context**: E0 (each sentence alone, no BiLSTM, no CRF) 0.361; E1 (same encoder and method, with context) 0.455.
- **Phase-1 backbone**: E1 on the original A4 0.427; on the cleaned 1B-token A4 0.455.

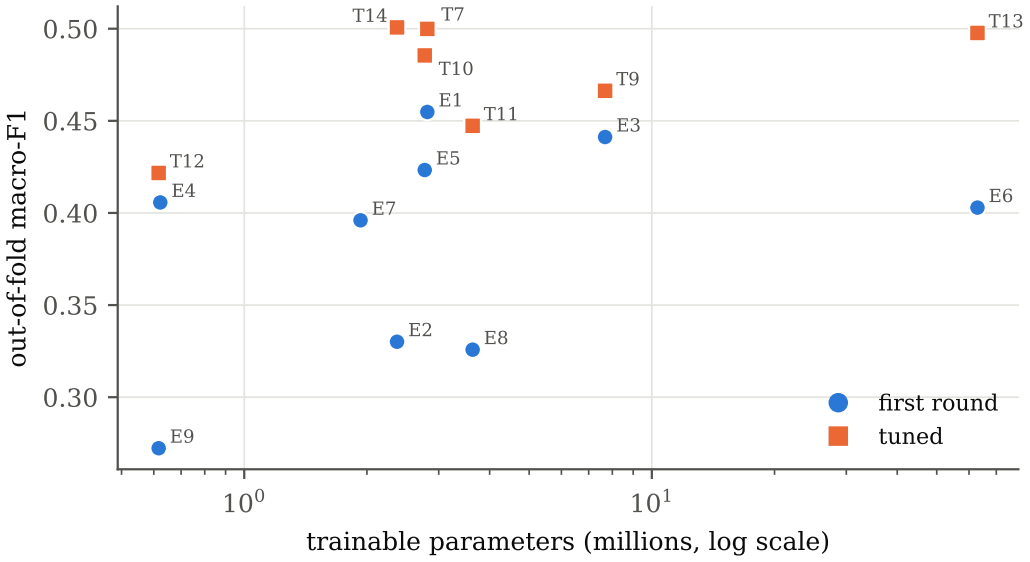


Figure 3: OOF macro-F1 against trained parameters (document layer included), first round and tuned runs.

5.2 Tuning rounds

Four rounds of follow-up runs, each defined by a config that inherits from an earlier experiment. The "purpose" is the comment written in the config before the run; the "change" is the computed difference between the two configs; Δ is the OOF difference against the parent.

Table 4: Tuning runs.

Run	Parent	Purpose	Change	OOF F1	Δ	Fold mean
T1	E1	E1 with 512-sentence training chunks (the BiLSTM sees most of a document; targets Dissent, rulings)	chunk_sents=512, chunks_per_step=1	0.416	-0.038	0.432±0.049
T2	E1	E1 with weaker logit adjustment (macro recall 0.51 > precision 0.44 at tau 1)	logit_adjust_tau=0.5	0.459	+0.004	0.509±0.033
T3	E3	E3 trained longer (best epoch was 12, 11, 12, 9 of 12)	epochs=20, patience=6	0.467	+0.026	0.475±0.022
T4	E2	E2 with causal attention (A1 was pre-trained causally only)	attention=causal	0.477	+0.147	0.499±0.010
T6	E1	E1 with causal attention only (isolates the attention effect on A4)	attention=causal	0.459	+0.004	0.489±0.017
T7	E1	E1 (A4 LoRMA) causal + 20 epochs + tau 0.5	attention=causal, epochs=20, logit_adjust_tau=0.5, patience=6	0.500	+0.045	0.547±0.023
T8	E2	T4 (A1 LoRA causal) + 20 epochs + tau 0.5 (T4 peaked at its last epoch in 3/5 folds)	attention=causal, epochs=20, logit_adjust_tau=0.5, patience=6	0.491	+0.161	0.538±0.034
T9	E3	T3 (A3 rsLoRA, 20 epochs) + tau 0.5	epochs=20, logit_adjust_tau=0.5, patience=6	0.466	+0.025	0.519±0.026
T10	E5	E5 (A4 QLoRA) causal + 20 epochs + tau 0.5	attention=causal, epochs=20, logit_adjust_tau=0.5, patience=6	0.485	+0.062	0.544±0.030
T11	E8	E8 (A4 prefix tuning) causal + 20 epochs + tau 0.5	attention=causal, epochs=20, logit_adjust_tau=0.5, patience=6	0.447	+0.121	0.504±0.035
T12	E9	E9 (A1 prompt tuning) causal + 20 epochs + tau 0.5	attention=causal, epochs=20, logit_adjust_tau=0.5, patience=6	0.422	+0.149	0.456±0.033

Run	Parent	Purpose	Change	OOF F1	Δ	Fold mean
T13	E6	E6 (KD, A4 6/10-layer student, full FT) with the stronger T7 teacher, causal, 20 epochs, tau 0.5	attention=causal, epochs=20, logit_adjust_tau=0.5, patience=6, teacher=T7	0.498	+0.095	0.537±0.026
T14	T8	T8 with 1024-token windows (A1 was pre-trained on 1024-token blocks)	max_windows=16, window=1024, max_windows=8, window=1024	0.501	+0.009	0.558±0.028
T15	T8	T8 with LoRA rank 32 (alpha 64, same scale)	alpha=64, r=32	0.493	+0.001	0.548±0.021
T16	T8	T8 with half the learning rate (fold 3 peaked at epoch 3)	lr=0.0005	0.497	+0.005	0.552±0.022
T17	T8	T8 with a wider document layer (BiLSTM hidden 512)	hidden=512	0.500	+0.008	0.557±0.025
T18	T8	T8, another seed (same folds) – for a seed ensemble	seed=2024	0.500	+0.008	0.548±0.022
T19	T8	T8, a third seed (same folds) – for a seed ensemble	seed=7	0.500	+0.008	0.553±0.026

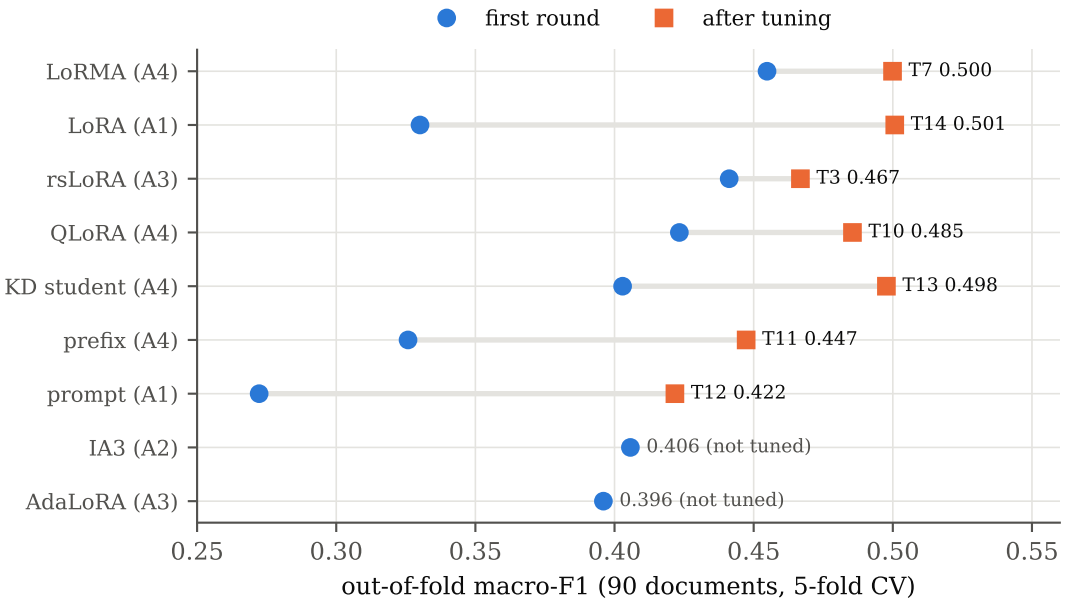


Figure 4: Each technique's first-round OOF macro-F1 and its best tuned version.

5.3 Per-label results and confusions

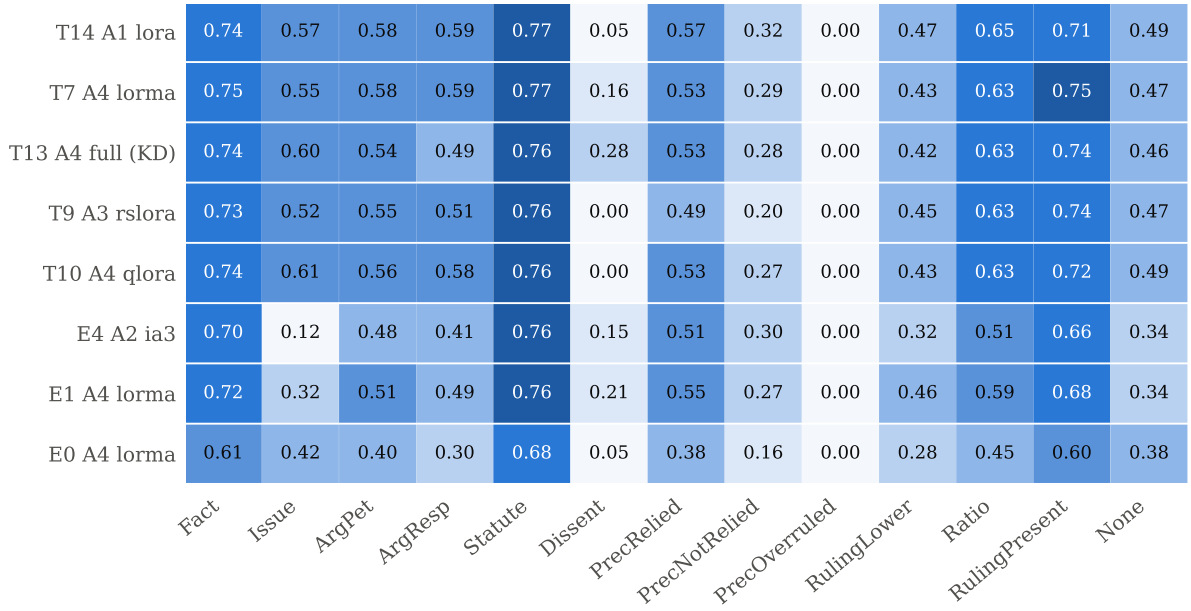


Figure 5: OOF F1 per label for the best tuned runs and three first-round references.

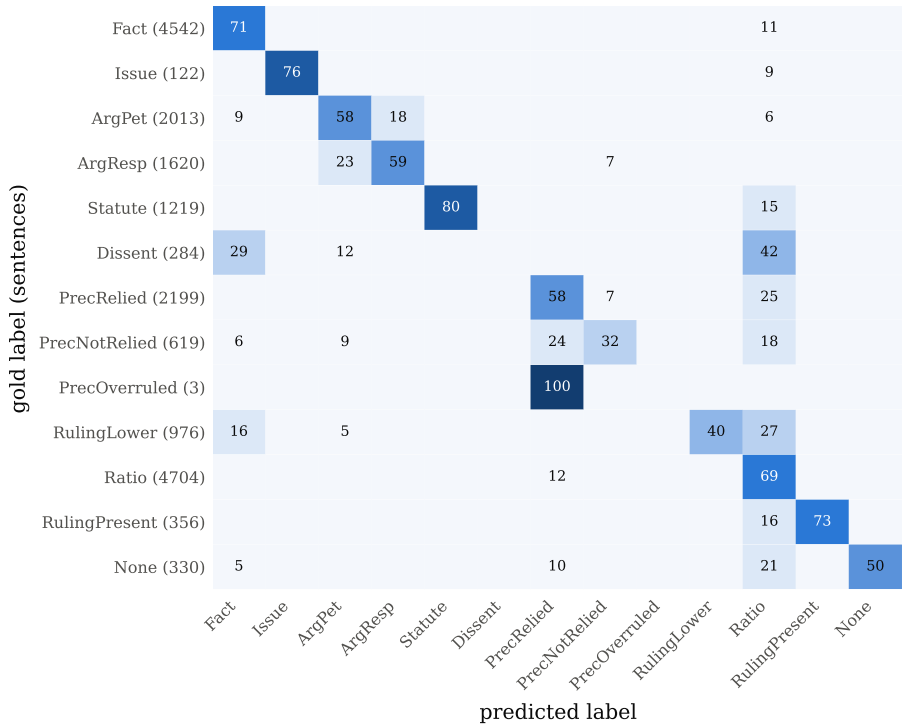


Figure 6: Row-normalised OOF confusion matrix of T14 (A1 LoRA), in percent of each gold label; cells below 5% are left blank.

Largest confusions in sentences (gold → predicted, T14): Ratio→PrecRelied 548; PrecRelied→Ratio 539; Fact→Ratio 499; ArgPet→ArgResp 370; ArgResp→ArgPet 367; RulingLower→Ratio 268.

5.4 Ensembles, calibration and post-processing

Table 5: Cross-run ensembles and calibration (OOF macro-F1; calibrated = nested-CV estimate).

Name	Members	OOF F1	CL	IT	Calibrated	Δ
T5	E1, E3	0.467	0.479	0.481	–	–
C0	T4	0.477	0.496	0.451	0.492	+0.016
C4	T8	0.491	0.526	0.505	0.482	-0.010
C1	T7, T8	0.507	0.525	0.532	0.511	+0.004
C2	T7, T8, T10	0.509	0.523	0.539	0.504	-0.005
C3	T7, T8, T10, T9	0.511	0.521	0.541	0.512	+0.001

For the C3 ensemble the predicted label sequence has 32.5 segments per document on average, against 28.2 in gold. On the simulated 10-document test sets C3 scores 0.585 on average, with a 10th–90th percentile range of 0.522–0.649.

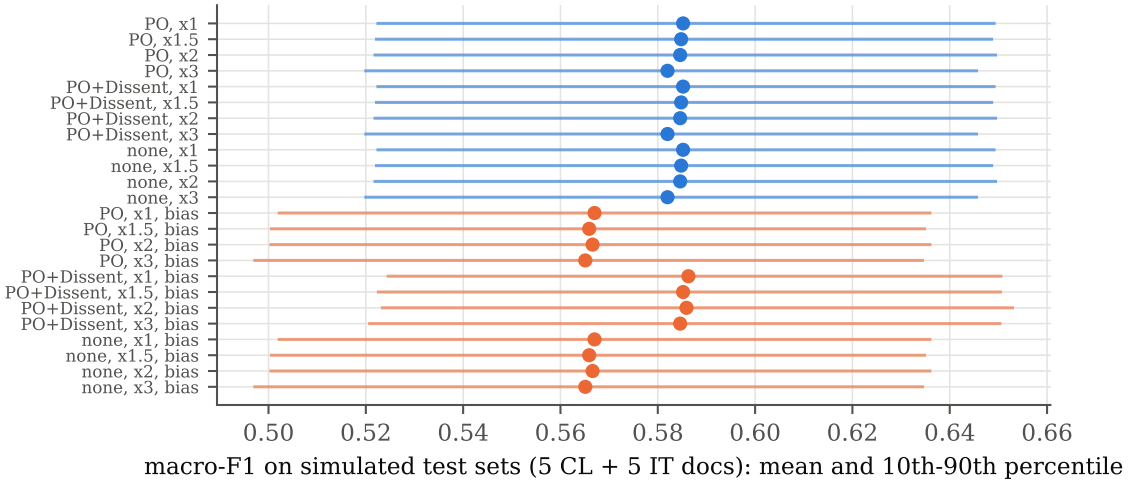


Figure 7: C3 decoding variants on 3,000 simulated test sets (blocked labels: PO = PrecedentOverruled; x = CRF transition scale; bias = calibrated per-label bias).

Table 6: C3 decoding variants: OOF macro-F1 over 90 documents, simulated-test mean and 10th–90th percentile, predicted segments per document.

Blocked	Scale	Bias	OOF F1	Test-like mean	p10–p90	Segments
PO	1	no	0.5113	0.5852	0.522–0.649	32.5
PO	1	yes	0.5335	0.5670	0.502–0.636	33.6
PO	1.5	no	0.5113	0.5848	0.522–0.649	30.4
PO	1.5	yes	0.5326	0.5659	0.501–0.635	31.3
PO	2	no	0.5113	0.5846	0.522–0.649	28.6
PO	2	yes	0.5330	0.5666	0.500–0.636	29.4
PO	3	no	0.5100	0.5820	0.520–0.645	26.2
PO	3	yes	0.5313	0.5651	0.497–0.634	26.6
PO+Dissent	1	no	0.5113	0.5852	0.522–0.649	32.5
PO+Dissent	1	yes	0.5157	0.5863	0.525–0.650	33.2
PO+Dissent	1.5	no	0.5113	0.5848	0.522–0.649	30.4
PO+Dissent	1.5	yes	0.5148	0.5852	0.523–0.650	31.0
PO+Dissent	2	no	0.5113	0.5846	0.522–0.649	28.6
PO+Dissent	2	yes	0.5156	0.5859	0.523–0.653	29.2
PO+Dissent	3	no	0.5100	0.5820	0.520–0.645	26.2
PO+Dissent	3	yes	0.5147	0.5846	0.521–0.650	26.4
none	1	no	0.5113	0.5852	0.522–0.649	32.5
none	1	yes	0.5335	0.5670	0.502–0.636	33.6
none	1.5	no	0.5113	0.5848	0.522–0.649	30.4
none	1.5	yes	0.5326	0.5659	0.501–0.635	31.3
none	2	no	0.5113	0.5846	0.522–0.649	28.6
none	2	yes	0.5330	0.5666	0.500–0.636	29.4
none	3	no	0.5100	0.5820	0.520–0.645	26.2
none	3	yes	0.5313	0.5651	0.497–0.634	26.6

5.5 Submissions

35 format-checked zips are in `p3/submissions/` (Table 7).

To be written by the author — leaderboard scores

Add the Codabench test macro-F1 (and precision / recall) of each uploaded submission, and say which six or more count towards the requirement.

Table 7: Submission files and the run or ensemble they come from.

File	Run
`C1_ens_t7_t8_20261015_113934.zip`	C1
`C2_ens_t7_t8_t10_20261015_114255.zip`	C2
`C3_ens_t7_t8_t10_t9_20261015_114644.zip`	C3
`C3_ens_t7_t8_t10_t9_pp_blkPO+Dissent_s1_bias_20261015_225701.zip`	C3
`C4_t8_cal_20261015_122824.zip`	C4
`E0_a4_lorma_sentonly_20261015_033056.zip`	E0
`E1_a4_lorma_20261015_033112.zip`	E1
`E1_a4_lorma_origP1_20261015_033130.zip`	E1
`E2_a1_lora_20261015_033147.zip`	E2
`E3_a3_rslora_20261015_033204.zip`	E3
`E4_a2_ia3_20261015_033221.zip`	E4
`E5_a4_qlora_20261015_033239.zip`	E5
`E6_kd_a4_small_20261015_033255.zip`	E6
`E7_a3_adalora_20261015_033313.zip`	E7
`E8_a4_prefix_20261015_033330.zip`	E8
`E9_a1_prompt_20261015_033348.zip`	E9
`T10_e5_qlora_causal_20261015_113125.zip`	T10
`T11_e8_prefix_causal_20261015_113202.zip`	T11
`T12_e9_prompt_causal_20261015_113239.zip`	T12
`T13_kd_t7_20261015_114856.zip`	T13
`T14_t8_win1024_20261015_130813.zip`	T14
`T15_t8_r32_20261015_130815.zip`	T15
`T16_t8_lr5e-4_20261015_130818.zip`	T16

File	Run
T17_t8_hidden512_20261015_130820.zip	T17
T18_t8_seed2_20261015_130822.zip	T18
T19_t8_seed3_20261015_130825.zip	T19
T1_e1_longchunk_20261015_034751.zip	T1
T2_e1_tau05_20261015_034802.zip	T2
T3_e3_ep20_20261015_034805.zip	T3
T4_e2_causal_20261015_034807.zip	T4
T5_ens_e1_e3_20261015_033413.zip	T5
T6_e1_causal_20261015_113312.zip	T6
T7_e1_causal_ep20_tau05_20261015_113338.zip	T7
T8_e2_causal_ep20_tau05_20261015_113406.zip	T8
T9_e3_ep20_tau05_20261015_113444.zip	T9

6 Analysis

To be written by the author — analysis of the results

The course rules do not allow LLM-written result analysis. Questions the course asks (§9.5), with the evidence above:

- Which adaptation techniques work best for RR, and can parameter-efficient methods approach full adaptation (Table 3, Table 4, Figure 3)?
- Does context improve sentence-level prediction (E0 vs E1)?
- Which roles remain hard to distinguish, and why (Figure 5, Figure 6)?
- Does performance differ between CL and IT?
- Does distillation keep performance while improving efficiency (E6, T13)?
- How do model size and compute change across experiments?
- Are the Phase-1 representations useful here (original vs cleaned backbone; causal vs bidirectional)?
- Why can a 10-document test score differ from the cross-validated estimate (Figure 7)?

7 Limitations and next steps

Facts that limit the conclusions (the interpretation is the author's):

- 90 labelled documents; fold scores vary by 0.01–0.05 (standard deviation).
- Several tuning decisions were made on the same out-of-fold scores they are reported on; only the calibration estimate is nested.
- One seed per run except T8, T18 and T19 (same configuration, three seeds).
- PrecedentOverruled (3 sentences) and Dissent (2 documents) have too few examples to be learned.
- Training chunks hold 128 sentences, shorter than most documents; the 512-sentence variant (T1) was not combined with the later changes.

To be written by the author — what you would do next

With more time or compute: ...

8 Conclusions

To be written by the author — conclusions

Summarise what you conclude from the experiments, in your own words.

9 Reproducibility

First round (CV + zips)	`bash run_all.sh`
Tuning rounds	`bash run_tune.sh` (configs in `configs/tune*`)
One experiment	`python train.py -config configs/e1_a4_lorma.yaml [-folds 0 2]`
Fold-ensemble zip	`python make_submission.py -run runs/T7_e1_causal_ep20_tau05`
Cross-run ensemble (OOF)	`python ensemble_cv.py -run runs/E1_a4_lorma runs/E3_a3_rslora -name T5`
Ensemble + calibration + zip	`python calibrate.py -run runs/T7... runs/T8... -name C1_ens_t7_t8`
Post-processing study	`python postproc.py -name C3_ens_t7_t8_t10_t9 -use-bias`
Comparison table	`python compare.py`
Figures and tables	`python p3/report/make_figures.py`

To be written by the author — repository link

GitHub: `https://github.com/...`

LLM declaration

To be written by the author — LLM declaration (mandatory)

Fill in the course questionnaire. Describe honestly which parts used an LLM assistant (for example the model and PEFT code, CV/ensemble/calibration scripts, figures and this report's methods/results text) and which parts are your own (design decisions, hyperparameters, analysis, conclusions).

References

[1] P. Bhattacharya et al. Identification of rhetorical roles of sentences in Indian legal judgments. JURIX 2019.

[2] H. Bihany, S. Patel, A. Modi. LoRMA: Low-rank multiplicative adaptation for LLMs. Findings of ACL 2025.

[3] T. Dettmers et al. QLoRA: Efficient finetuning of quantized LLMs. NeurIPS 2023.

[4] G. Hinton, O. Vinyals, J. Dean. Distilling the knowledge in a neural network. arXiv:1503.02531, 2015.

[5] S. Hochreiter, J. Schmidhuber. Long short-term memory. Neural Computation, 1997.

[6] E. Hu et al. LoRA: Low-rank adaptation of large language models. ICLR 2022.

[7] D. Kalajdzievski. A rank stabilization scaling factor for fine-tuning with LoRA. arXiv:2312.03732, 2023.

[8] J. Lafferty, A. McCallum, F. Pereira. Conditional random fields. ICML 2001.

[9] B. Lester, R. Al-Rfou, N. Constant. The power of scale for parameter-efficient prompt tuning. EMNLP 2021.

[10] X. Li, P. Liang. Prefix-tuning: Optimizing continuous prompts for generation. ACL 2021.

[11] H. Liu et al. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. NeurIPS 2022.

[12] V. Malik et al. Semantic segmentation of legal documents via rhetorical roles. NLLP Workshop 2022.

[13] A. Menon et al. Long-tail learning via logit adjustment. ICLR 2021.

[14] Q. Zhang et al. AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. ICLR 2023.